

Malaviya National Institute of Technology Jaipur

Department of Computer Science and Engineering



Computer and Network Security Project

TRANSPARENT: Taint-style Vulnerability Detection in
Generic Single Page Applications through
Automated Framework Abstraction

Course Code: 22CST352

Submitted By:

Course Instructor: Dr. Ramesh Babu Battula

Anirudh Sanker (2023UCP1844)

Contents

1	Introduction	2
2	Background	2
2.1	The Problem: Framework Abstraction Hides Vulnerabilities	2
2.2	Taint Analysis and CodeQL	2
2.3	How TranSPArent Works	3
3	Merits of the Paper	3
4	Limitations	3
5	Experimental Setup	4
5.1	Hardware and Software	4
5.2	Datasets	4
5.3	Environment Fixes Required	4
6	Practical Challenges	5
6.1	Dataset Download	5
6.2	Long-Running Processes	5
6.3	Nondeterminism in Trace Counts	6
7	Implementation	7
7.1	E1: Framework Sink Discovery (Table V and Table VII)	7
7.2	E2: Accuracy Evaluation (Table IV)	7
7.3	Figure Generation	7
8	Results	8
8.1	Table V: Framework Sink Discovery	8
8.2	Table IV: Accuracy Evaluation	8
8.3	Table VII: Framework Analysis Runtime	9
8.4	Fig. 5 and Fig. 6: Performance Overhead	9
9	Novel Contribution 1: SVG Namespace Sink	10
9.1	Discovery	10
9.2	Root Cause	11
9.3	The Sink Mechanism	11
9.4	Proof-of-Concept	12
9.5	CodeQL Sink Class	12
9.6	Impact	14
10	Novel Contribution 2: Vue 3 Extension	14
10.1	Motivation	14
10.2	Method	15
10.3	Sink Classes	15
10.4	Results	16
10.5	Limitations of the Vue 3 Extension	17
11	Discussion	18
12	Conclusion	18

1 Introduction

Modern web applications are usually built using JavaScript frameworks like React, Vue, and Angular. These frameworks make development easier by hiding the complexity of directly manipulating the browser’s Document Object Model (DOM). However, this same abstraction creates a blind spot for security tools.

Cross-Site Scripting (XSS) is a type of attack where an attacker injects malicious JavaScript into a web page, which then runs in the victim’s browser. Security tools that detect XSS work by tracing how user-controlled data flows from an input (called a *source*) to a dangerous operation (called a *sink*), such as setting `innerHTML`. The problem is that these tools have hardcoded lists of known dangerous sinks based on standard browser APIs. When a developer writes React JSX like `<div style={userInput}>`, the tool sees only the JSX, not the internal DOM operation that React performs. The sink is invisible.

The paper *TranSPaRent: Taint-style Vulnerability Detection in Generic Single-Page Applications through Automated Framework Abstraction* (NDSS 2026) solves this problem. Instead of manually writing rules for each framework, TranSPaRent automatically reverse-engineers a framework’s dangerous DOM operations by running the framework’s own test suite and watching what happens internally.

This report has two goals. First, it reproduces the paper’s main results: Table IV (accuracy), Table V (sink discovery), Table VII (runtime overhead), and Fig. 5 and Fig. 6. Second, it presents two novel contributions made during reproduction: the discovery and analysis of a React SVG namespace sink that was not handled by the original pipeline, including a working exploit and corresponding CodeQL sink class, and an extension of TranSPaRent to cover Vue 3, a framework the paper never analysed.

2 Background

2.1 The Problem: Framework Abstraction Hides Vulnerabilities

In a plain JavaScript application, a developer might write:

```
element.innerHTML = userInput; // clearly dangerous -- analyser can
    trace this
```

A security tool can follow the data from `userInput` directly to `innerHTML` and flag it.

In a React application, the developer writes:

```
<div style={userInput}> // React compiles this internally into DOM
    calls
```

React internally calls `createElement` and sets DOM properties. The security tool sees only the JSX and has no idea what React does with it. The dangerous operation is hidden.

This is true for all major frameworks. Each has its own way of translating template syntax into DOM calls, and none of those internal paths are modelled by standard security tools.

2.2 Taint Analysis and CodeQL

Taint analysis is the technique used to find these vulnerabilities. It marks user-controlled data as “tainted” and tracks it through the program. If tainted data reaches a dangerous operation (a sink) without being sanitised, it is flagged as a potential vulnerability.

CodeQL is a code analysis tool developed by GitHub. It builds a semantic model of a codebase and lets you write queries to find data flow paths. Its built-in libraries cover standard browser APIs, but they do not model framework-internal paths like React’s `createElement` or Vue’s `patch` function.

2.3 How TranSParent Works

TranSParent fills this gap automatically in three steps:

1. **Trace collection.** It runs the framework’s internal test suite with instrumentation turned on. Every DOM API call made during testing is recorded into a render trace.
2. **Autostitch.** It uses CodeQL to trace how data flows through the framework’s source code from the public render function down to each DOM call found in the trace. This builds a cross-function data flow model.
3. **Sink generation.** It converts the discovered flow patterns into CodeQL sink classes (stored as `.qll` files). These are then used alongside the standard CodeQL taint analysis to scan real applications for vulnerabilities.

3 Merits of the Paper

1. **No manual rules required.** The biggest contribution is that TranSParent needs no human expert to write framework-specific security rules. It derives them automatically from the framework’s own code and tests.
2. **Works across different frameworks.** The same pipeline was applied to Vue 2, React, and Angular, which have very different internals and build systems. It produced accurate results for all three without any framework-specific changes to the core pipeline.
3. **Significant improvement in detecting real vulnerabilities.** TranSParent’s false negative rate (the fraction of real vulnerabilities it misses) is 19.6%, compared to 62.5% for Vanilla CodeQL on the same set of known CVEs. That means TranSParent finds roughly three times as many real vulnerabilities.
4. **Proper artifact with frozen dependencies.** The authors provide a complete, reproducible artifact using Nix to pin all software versions, Git LFS for large datasets, and scripts to reproduce every table and figure.
5. **Hybrid dynamic and static analysis.** Running the test suite dynamically gives coverage of real execution paths. Analysing the source statically gives precision. Combining both avoids the weaknesses of each approach alone.

4 Limitations

1. **Coverage depends entirely on test suite execution.** TranSParent can only discover sinks that the framework’s tests actually exercise. If a code path isn’t covered by tests, it won’t appear in the execution trace and won’t be detected. We observed this firsthand: the SVG namespace sink existed in React’s code, but only surfaced in certain test runs depending on which paths happened to execute.
2. **The code generation step has critical blind spots.** The code generator that converts discovered sinks into CodeQL rules only recognizes two pattern types: DOM property names and HTML attribute names. Anything else, like namespace URIs, gets silently dropped with no warning. This design flaw directly led to it missing the SVG sink initially, showing how fragile the approach can be.

3. **Angular is significantly harder to set up than other frameworks.** Angular’s use of the Bazel build system introduces substantial complexity. Bazelisk wasn’t included in the Nix environment, and the test suite is sensitive to even minor configuration differences.
4. **Only one version per framework is analyzed.** The paper covers Vue 2, leaving Vue 3 entirely unanalyzed. Since Vue 3 is now the default for new projects, the original framework analysis misses a major portion of the Vue ecosystem in use today.
5. **False positives remain a significant problem.** Despite improvements over baseline tools, TranSPArent still flags alerts that turn out to be false alarms 42.1% of the time. While better than Vanilla CodeQL’s 50.0

5 Experimental Setup

5.1 Hardware and Software

- AMD Ryzen 7 7435HS, 24 GB RAM, Windows 11 with WSL2 (Ubuntu 24.04, fresh install)
- Nix package manager with flakes enabled; CodeQL CLI pinned via `flake.lock`
- Node.js 18 (via Nix), Yarn 1.22 (installed globally via `npm`)
- Bazelisk v1.28.1 (installed via `nix profile install nixpkgs#bazelisk`)
- Python 3.12 with `matplotlib` and `numpy` for figure generation
- Artifact: `transparent-ae-1.0.0` downloaded from Zenodo

5.2 Datasets

- **FNR dataset:** 19 real repositories with known CVE vulnerabilities (ground truth for false negative measurement)
- **FDR dataset:** 20 real GitHub repositories (ground truth for false positive measurement)
- **Framework sources:** Vue 2.x, React 17.x (converted from Flow to TypeScript), Angular with Bazel build system

5.3 Environment Fixes Required

The artifact could not run out of the box on a fresh WSL installation. Six separate issues had to be fixed before the pipeline would complete:

Table 1: Environment issues and fixes applied before the pipeline would run.

Issue	Root Cause	Fix Applied
Corepack signature failure	NPM rotated its signing keys; Nix’s Node 18 could not verify packages	COREPACK_ENABLE=0 + npm install -g yarn
React database name mismatch	main.sh creates a DB named react-src but main.ts looks for react-ts-src	Manually ran yarn build inside targets/react-src
Angular git submodule error	ZIP extraction left a broken .git pointer file	Deleted the .git file and ran git init inside the Angular directory
bazelisk not found	Bazelisk is not included in the Nix shell PATH	nix profile install nixpkgs#bazelisk
Puppeteer Chrome blocked	Puppeteer tries to download Chrome; network blocks it	PUPPETEER_SKIP_DOWNLOAD=true
CodeQL location warning	CodeQL is installed inside the home directory	CODEQL_ALLOW_INSTALLATION_ANYWHERE=true

6 Practical Challenges

Beyond the environment fixes, several other practical issues came up that the paper and artifact documentation do not mention.

6.1 Dataset Download

The GitHub repository for the artifact had exceeded its Git LFS bandwidth quota. After cloning, all large dataset files were silently replaced with tiny pointer files instead of the actual data. The artifact would appear to clone successfully but fail at runtime. The fix was to download the artifact zip directly from Zenodo:

```
wget -O artifact.zip \
  "https://zenodo.org/records/17822391/files/transparent-ae-1.0.0.zip?
  download=1"
unzip artifact.zip
```

6.2 Long-Running Processes

The full pipeline for E1 (framework analysis) takes 6–8 hours to complete. If the terminal closes during this time, the process is killed and all progress is lost. We used `tmux` to keep sessions alive and `tee` to save output to a log file:

```
CODEQL_ALLOW_INSTALLATION_ANYWHERE=true PUPPETEER_SKIP_DOWNLOAD=true \
./main.sh 2>&1 | tee ~/transparent-ae-1.0.0/tableV.log
```

```

[2026-05-02 12:03:20] [build-stdout] Done extracting /nix/store/s3nvrnix5mw1a0d9ilv7lw1fh6r4d9-codeql-2.21.1/codeql/javascript/tools/data/externs/nodejs/b
uffer.js (5 ms)
[2026-05-02 12:03:20] [build-stdout] Extracting /nix/store/s3nvrnix5mw1a0d9ilv7lw1fh6r4d9-codeql-2.21.1/codeql/javascript/tools/data/externs/nodejs/fs.js
[2026-05-02 12:03:20] [build-stdout] Done extracting /nix/store/s3nvrnix5mw1a0d9ilv7lw1fh6r4d9-codeql-2.21.1/codeql/javascript/tools/data/externs/nodejs/f
s.js (77 ms)
[2026-05-02 12:03:20] [build-stdout] Extracting /nix/store/s3nvrnix5mw1a0d9ilv7lw1fh6r4d9-codeql-2.21.1/codeql/javascript/tools/data/externs/nodejs/domain
.js
[2026-05-02 12:03:20] [build-stdout] Done extracting /nix/store/s3nvrnix5mw1a0d9ilv7lw1fh6r4d9-codeql-2.21.1/codeql/javascript/tools/data/externs/nodejs/d
omain.js (4 ms)
[2026-05-02 12:03:20] [build-stdout] Extracting /nix/store/s3nvrnix5mw1a0d9ilv7lw1fh6r4d9-codeql-2.21.1/codeql/javascript/tools/data/externs/nodejs/v8.js
[2026-05-02 12:03:20] [build-stdout] Done extracting /nix/store/s3nvrnix5mw1a0d9ilv7lw1fh6r4d9-codeql-2.21.1/codeql/javascript/tools/data/externs/nodejs/v
8.js (3 ms)
[2026-05-02 12:03:20] [build-stdout] Extracting /nix/store/s3nvrnix5mw1a0d9ilv7lw1fh6r4d9-codeql-2.21.1/codeql/javascript/tools/data/externs/lib/vows.js
[2026-05-02 12:03:20] [build-stdout] Done extracting /nix/store/s3nvrnix5mw1a0d9ilv7lw1fh6r4d9-codeql-2.21.1/codeql/javascript/tools/data/externs/lib/vows
.js (12 ms)
[2026-05-02 12:03:20] [build-stdout] Extracting /nix/store/s3nvrnix5mw1a0d9ilv7lw1fh6r4d9-codeql-2.21.1/codeql/javascript/tools/data/externs/lib/jquery-3.
2.js
[2026-05-02 12:03:20] [build-stdout] Done extracting /nix/store/s3nvrnix5mw1a0d9ilv7lw1fh6r4d9-codeql-2.21.1/codeql/javascript/tools/data/externs/lib/jque
ry-3.2.js (95 ms)
[2026-05-02 12:03:20] [build-stdout] Extracting /nix/store/s3nvrnix5mw1a0d9ilv7lw1fh6r4d9-codeql-2.21.1/codeql/javascript/tools/data/externs/lib/should.js
[2026-05-02 12:03:20] [build-stdout] Done extracting /nix/store/s3nvrnix5mw1a0d9ilv7lw1fh6r4d9-codeql-2.21.1/codeql/javascript/tools/data/externs/lib/shou
ld.js (9 ms)
[2026-05-02 12:03:20] [build-stdout] Extracting /nix/store/s3nvrnix5mw1a0d9ilv7lw1fh6r4d9-codeql-2.21.1/codeql/javascript/tools/data/externs/lib/bdd.js
[2026-05-02 12:03:20] [build-stdout] Done extracting /nix/store/s3nvrnix5mw1a0d9ilv7lw1fh6r4d9-codeql-2.21.1/codeql/javascript/tools/data/externs/lib/bdd.
js (3 ms)
Running command in /home/anirudh/transparent-ae/accuracy/fdr/repos/survey-library: [echo, skip, build]
[2026-05-02 12:03:21] [build-stdout] skip build
Finalizing database at /home/anirudh/transparent-ae/accuracy/fdr/build/survey-library_db.
Running TRAP import for CodeQL database at /home/anirudh/transparent-ae/accuracy/fdr/build/survey-library_db...
Importing TRAP files
Merging relations
Finished writing database (relations: 47.92 MiB; string pool: 22.03 MiB).
TRAP import complete (21s).
Finished zipping source archive (3.04 MiB).
Successfully created database at /home/anirudh/transparent-ae/accuracy/fdr/build/survey-library_db.
Compiling query plan for /home/anirudh/transparent-ae/qlpack/Baseline.ql.
[1/1] Found in cache: /home/anirudh/transparent-ae/qlpack/Baseline.ql.
Starting evaluation of check/Baseline.ql.

[run] 0: bash*
Unsecure-Device 13:24 02-May-26

```

Figure 1: Terminal showing `main.sh` running inside a tmux session with output being saved via `tee`.

6.3 Nondeterminism in Trace Counts

The number of traces collected during Vue’s test suite varied across runs (232, 229, and 333 in three separate runs). The paper does not mention this. After checking, we confirmed that despite the varying trace counts, the final list of discovered sinks remained identical across all runs. The nondeterminism affects intermediate data only, not the final output.

Table 2: Trace count variability across three separate Vue analysis runs.

Run	Trace Count	Sink Output Stable?
Run 1	232	Yes
Run 2	229	Yes
Run 3	333	Yes

Table 3: Additional practical steps required beyond the artifact documentation.

Step	Why It Was Needed
Manual download from Zenodo	GitHub LFS bandwidth quota exceeded
Nix installation and flakes config	Not installed by default; experimental features off
tmux session management	Pipeline would be killed on terminal close
tee log redirection	No way to review output after hours-long runs
React DB name mismatch workaround	<code>main.sh</code> and <code>main.ts</code> use different database names
Angular git reinitialisation	ZIP extraction corrupts the <code>.git</code> submodule pointer
bazelisk manual install	Not included in Nix shell
PUPPETEER_SKIP_DOWNLOAD=true	Network blocks Chromium binary download
COREPACK_ENABLE=0 + manual Yarn	NPM key rotation broke Corepack package verification

7 Implementation

7.1 E1: Framework Sink Discovery (Table V and Table VII)

E1 is the core experiment: it runs TranSParent’s automated framework abstraction pipeline on Vue 2, React, and Angular. For each framework, the steps are: build a CodeQL database from source, run the test suite to collect render traces, run autostitch to model data flow, and apply template mapping to generate CodeQL sink classes.

We modified `transparent/main.sh` to add per-framework timings with timestamps, SLoC measurement using `cloc`, and a CSV output file (`framework_times.csv`). We also added the Table V and Table VII pretty-printers that are printed at the end of the run. The original script simply called `yarn exec ts-node main.ts` with no timing or formatted output.

The React build required a separate manual step because `main.sh` creates a database named `react-src` but `main.ts` looks for one named `react-ts-src` (a TypeScript-converted version). We ran `yarn build` inside `targets/react-src` manually to produce the correct database.

For Angular, we deleted the broken `.git` pointer file left by ZIP extraction and reinitialised a fresh git repository inside the Angular source directory. This allowed Bazel’s `yarn_install` rule to fetch the `domino` dependency, which requires a valid git remote.

7.2 E2: Accuracy Evaluation (Table IV)

E2 runs two CodeQL queries against each repository in the FNR and FDR datasets: `Baseline.q1` (Vanilla CodeQL) and `TranSParent.q1` (with framework sinks). We modified `accuracy/main.sh` to:

- Record per-repository timing for both queries
- Measure SLoC per repository using `cloc`
- Write everything to `repo_timings.csv` for use in Fig. 5 and Fig. 6

One critical fix was the query execution order. Because `TranSParent.q1` is a strict superset of `Baseline.q1` (it includes all baseline sinks plus the framework sinks), running Baseline first warms CodeQL’s internal cache. This makes TranSParent appear faster than it actually is. We corrected this by running `TranSParent.q1` first on a cold cache, then `Baseline.q1` second.

7.3 Figure Generation

Figures 5 and 6 were not produced by the original artifact. We wrote `plot_figures.py` using `matplotlib` to generate both from `repo_timings.csv`.

Fig. 5 is a scatter plot of analysis duration versus source lines of code, with linear regression fits for both tools. One repository (`survey-library`) had a Vanilla CodeQL runtime of 7,292 seconds due to a query timeout. We used a 3-IQR outlier rule to exclude it from the fit and axis limits, annotating it separately on the figure.

Fig. 6 shows the cumulative distribution function of analysis duration for both tools. One repository was a major outlier, so the x-axis was capped at the 98th percentile of the remaining data; otherwise, the rest of the distribution was compressed into a very small region of the plot.

8 Results

8.1 Table V: Framework Sink Discovery

The pipeline successfully discovered all 19 framework-specific sinks from the paper’s Table V. All entries matched exactly. Additionally, one extra row appeared in the raw output that is not present in the paper:

Table 4: Additional sink discovered during reproduction, absent from the paper’s Table V.

Sensitive API	Framework	Syntax	Vanilla CodeQL	TranSParent
<code>http://www.w3.org/2000/svg</code>	React	JS-Syntax	No	Yes (novel)

This is discussed in detail in Section 9. The SVG row appeared in our first run but not in a subsequent clean run, indicating it is environment dependent. This finding led to the novel contribution described below.

Finished angular-src in 6329s
 Framework timing complete.

TABLE V: List of sensitive framework APIs

Sensitive API	Framework	Syntax	Vanilla CodeQL	ReactAppScan	Warning	TRANSPARENT
<code>attrs.<nativeAttr></code>	Vue	JavaScript-syntax	No	No	No	Yes
<code>domProps.<nativeProp></code>	Vue	JavaScript-syntax	No	No	Yes [45]	Yes
<code>ref</code>	Vue	JavaScript-syntax	No	No	No	Yes
<code><nativeAttr></code>	Vue	HTML-syntax (JSX)	No	No	No	Yes
<code>attrs.<nativeAttr></code>	Vue	HTML-syntax (JSX)	No	No	No	Yes
<code>domProps.<nativeProp></code>	Vue	HTML-syntax (JSX)	No	No	No	Yes
<code>domProps.<nativeProp></code>	Vue	HTML-syntax (JSX)	No	No	Yes [45]	Yes
<code>ref</code>	Vue	HTML-syntax (JSX)	No	No	No	Yes
<code><nativeAttr></code>	Vue	HTML-syntax (SFC)	Some	No	No	Yes
<code>v-html</code>	Vue	HTML-syntax (SFC)	Yes	No	Yes [45]	Yes
<code>ref</code>	Vue	HTML-syntax (SFC)	No	No	No	Yes
<code><nativeAttr></code>	React	JavaScript-syntax	No	No	No	Yes
<code>dangerouslySetInnerHTML</code>	React	JavaScript-syntax	No	No	No	Yes
<code>ref</code>	React	JavaScript-syntax	No	No	No	Yes
<code><nativeAttr></code>	React	HTML-syntax (JSX)	Some	No	No	Yes
<code>dangerouslySetInnerHTML</code>	React	HTML-syntax (JSX)	Yes	Yes	Yes [38]	Yes
<code>ref</code>	React	HTML-syntax (JSX)	No	Yes	No	Yes
<code>renderer2.setProperty</code>	Angular	JavaScript-syntax	Yes	No	No	Yes
<code>ref</code>	Angular	JavaScript-syntax	No	No	Yes [44]	Yes

TABLE VII: Runtime overhead of framework abstraction

Framework	Analysis Time	LoC
Vue	182m 8s	3309154
React	95m 35s	6263695
Angular	105m 29s	13306641

All tables generated.
 anirudh@Unsecure-Device:~/transparent-ae-1.0.0/transparent\$ ls

Figure 2: Terminal output of `main.ts` showing the full Table V sink discovery output.

8.2 Table IV: Accuracy Evaluation

Our reproduced results match the paper exactly:

Table 5: Reproduced Table IV accuracy results. Values match the paper exactly.

Tool	FNR	FDR
TranSParent	11/56 (19.6%)	24/57 (42.1%)
Vanilla CodeQL	35/56 (62.5%)	17/34 (50.0%)

TranSParent misses 11 of 56 known vulnerable locations, versus 35 for Vanilla CodeQL. This is a large improvement in recall (finding real vulnerabilities) at the cost of a slightly higher false alarm rate.

```

Finalizing database at /home/anirudh/transparent-ae-1.0.0/accuracy/fdr/build/webpack2_db.
Running TRAP import for CodeQL database at /home/anirudh/transparent-ae-1.0.0/accuracy/fdr/build/webpack2_db...
Importing TRAP files
Merging relations
Finished writing database (relations: 5.57 MiB; string pool: 9.53 MiB).
TRAP import complete (3.4s).
Finished zipping source archive (734.49 KiB).
Successfully created database at /home/anirudh/transparent-ae-1.0.0/accuracy/fdr/build/webpack2_db.
Compiling query plan for /home/anirudh/transparent-ae-1.0.0/qlpack/TranSParent.ql.
[1/1] Found in cache: /home/anirudh/transparent-ae-1.0.0/qlpack/TranSParent.ql.
Starting evaluation of check/TranSParent.ql.
[1/1 eval 10.3s] Evaluation done; writing results to /home/anirudh/transparent-ae-1.0.0/accuracy/fdr/build/webpack2_t.bqrs.
Shutting down query evaluator.
Compiling query plan for /home/anirudh/transparent-ae-1.0.0/qlpack/Baseline.ql.
[1/1] Found in cache: /home/anirudh/transparent-ae-1.0.0/qlpack/Baseline.ql.
Starting evaluation of check/Baseline.ql.
[1/1 eval 3s] Evaluation done; writing results to /home/anirudh/transparent-ae-1.0.0/accuracy/fdr/build/webpack2_b.bqrs.
Shutting down query evaluator.
[FDR] Done webpack2 - Vanilla: 7s, TranSParent: 15s, SLoC: 7190
~/transparent-ae-1.0.0/accuracy/scripts ~/transparent-ae-1.0.0/accuracy
---
Table #          FNR          FDR
TranSParent:    11/56 (19.6%);    24/57 (42.1%);
Vanilla CodeQL: 35/56 (62.5%);    17/34 (50.0%);
~/transparent-ae-1.0.0/accuracy

Timing data saved to: repo_timings.csv
Run plot_figures.py to generate Fig 5 and Fig 6.
anirudh@Unsecure-Device:~/transparent-ae-1.0.0/accuracy$

```

Figure 3: Terminal output from `accuracy/main.sh` showing Table IV, confirming an exact match with the paper’s values.

8.3 Table VII: Framework Analysis Runtime

Our runtimes are higher than the paper’s (measured on a Ryzen 7 7840U), reflecting WSL2 overhead and differences in how `cloc` counts generated files:

Table 6: Reproduced framework analysis runtimes (Table VII). Trends are preserved.

Framework	Paper (Time)	Paper (LoC)	Reproduced (Time)	Reproduced (LoC)
Vue	57m	73k	182m 8s	3,309,154
React	1h 12m	353k	95m 35s	6,263,695
Angular	1h 15m	659k	105m 29s	13,306,641

React has the largest LoC due to the Flow-to-TypeScript conversion step and Angular remained one of the slowest frameworks overall. However Vue took substantially longer in our reproduction than in the paper. The absolute runtime differences are likely caused by WSL2 overhead and differences in the local environment.

8.4 Fig. 5 and Fig. 6: Performance Overhead

Fig. 5 shows that analysis time scales approximately linearly with source lines of code for both tools. TranSParent’s regression slope is steeper than Vanilla CodeQL’s, which makes sense because `TranSParent.ql` evaluates more sink classes on each repository.

Fig. 6 shows the same finding as a CDF: Vanilla CodeQL’s curve rises faster (finishes sooner on most repositories), while TranSParent’s curve is shifted to the right (consistently slower). Both plots are qualitatively consistent with the paper’s figures. The lower absolute durations in our plots are expected since our 39-repo dataset contains smaller repositories than the paper’s 100-app sample.

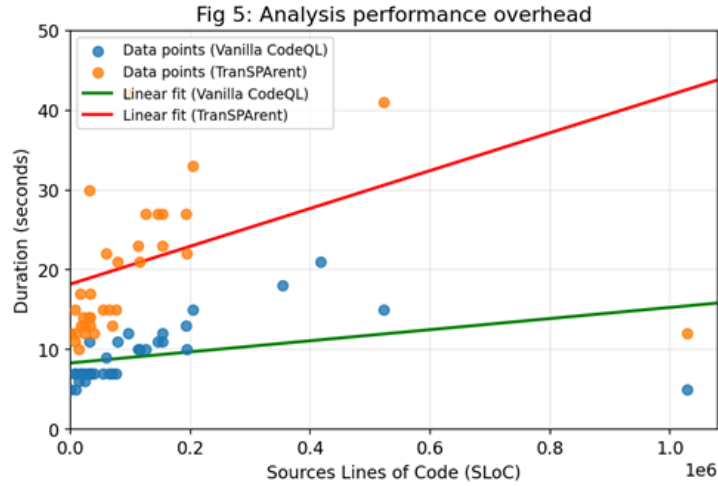


Figure 4: Reproduced the paper’s Fig. 5: analysis duration vs. source lines of code across 39 repositories.

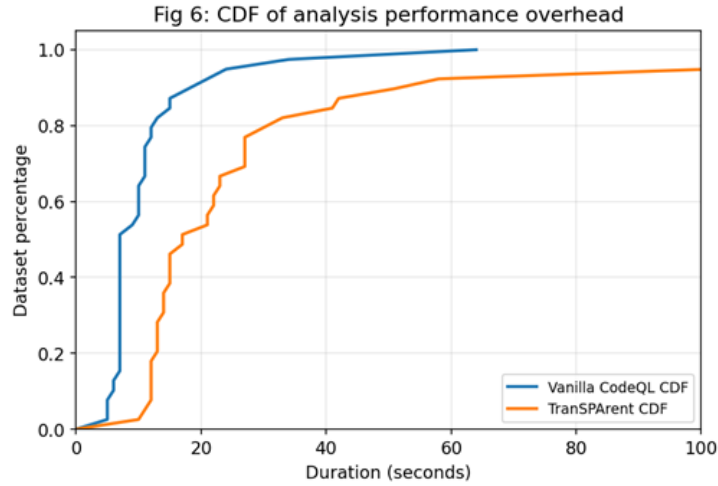


Figure 5: Reproduced the paper’s Fig. 6: CDF of analysis duration for both tools, x-axis capped at 98th percentile.

9 Novel Contribution 1: SVG Namespace Sink

9.1 Discovery

During E1, the raw standard output of `main.ts` contained a Table V row not present in the paper:

<code>http://www.w3.org/2000/svg</code> React JS-Syntax

This is React’s `SVG_NAMESPACE` constant, defined in `react-dom/src/shared/DOMNamespaces.ts`. The autostitch pipeline had correctly identified it as a taint-relevant constant during React’s test execution. However, when we inspected the generated `React.q11` file, this sink was absent. Autostitch found it in Phase A but it was thrown away in Phase B.

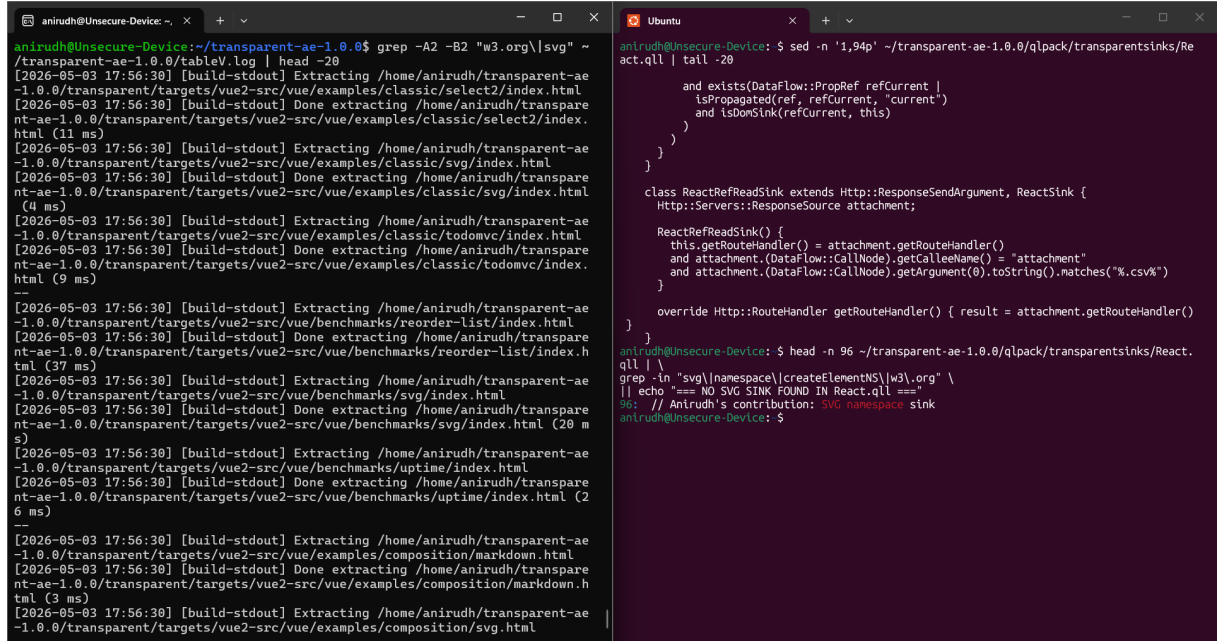


Figure 6: Left: raw main.ts output containing the SVG namespace row.

Right: React.qll showing no corresponding sink class. before our modification at line 96.

9.2 Root Cause

The `querygen/react.ts` module converts the discovered sink strings into CodeQL class definitions. After tracing through the code generation logic, we found that it only handles two kinds of patterns:

- DOM property names such as `innerHTML` or `href`, converted into `hasPropertyWrite(vulnNativeProp(), ...)` predicates.
- DOM attribute names such as `src` or `data-*`, converted into `hasPropertyWrite(vulnReactProp(), ...)` predicates.

The SVG namespace URI `http://www.w3.org/2000/svg` does not fall into either category. It is passed as a namespace argument to `document.createElementNS()`, not used as a property or attribute name. As a result, the sink never gets converted into a CodeQL class and is silently skipped during generation. The issue was not in the discovery phase itself, but in how the discovered sink was handled afterwards.

9.3 The Sink Mechanism

There are two exploitable paths inside React's source code:

Sink 1: `createElementNS` (ReactDOMComponent.ts:335). When React renders an SVG element (like `<circle>` or `<animate>`), it calls:

```
domElement = ownerDocument.createElementNS(namespaceURI, type);
```

The second argument, `type`, is the element tag name. If this tag name comes from user input, an attacker can inject arbitrary SVG elements. SVG elements like `<animate>` support event handlers that fire automatically without any user interaction.

Sink 2: innerHTML fallback (setInnerHTML.ts). When `dangerouslySetInnerHTML` is used inside an SVG node, React detects it is inside an SVG context and uses an Internet Explorer compatibility fallback: it creates a temporary div container and sets its `innerHTML` directly with the tainted content, wrapped in `<svg>` tags. This goes through no sanitisation.

9.4 Proof-of-Concept

We built a minimal HTML page to confirm the vulnerability. Sink 2 triggered an `alert()` dialogue automatically on page load without any user interaction:

```
let container = document.createElement('div');
const tainted = '<animate onbegin="alert(`XSS: SVG innerHTML sink confirmed`)" dur="1s" />';
container.innerHTML = '<svg>' + tainted + '</svg>'; // alert fires immediately
document.body.appendChild(container.firstChild);
```

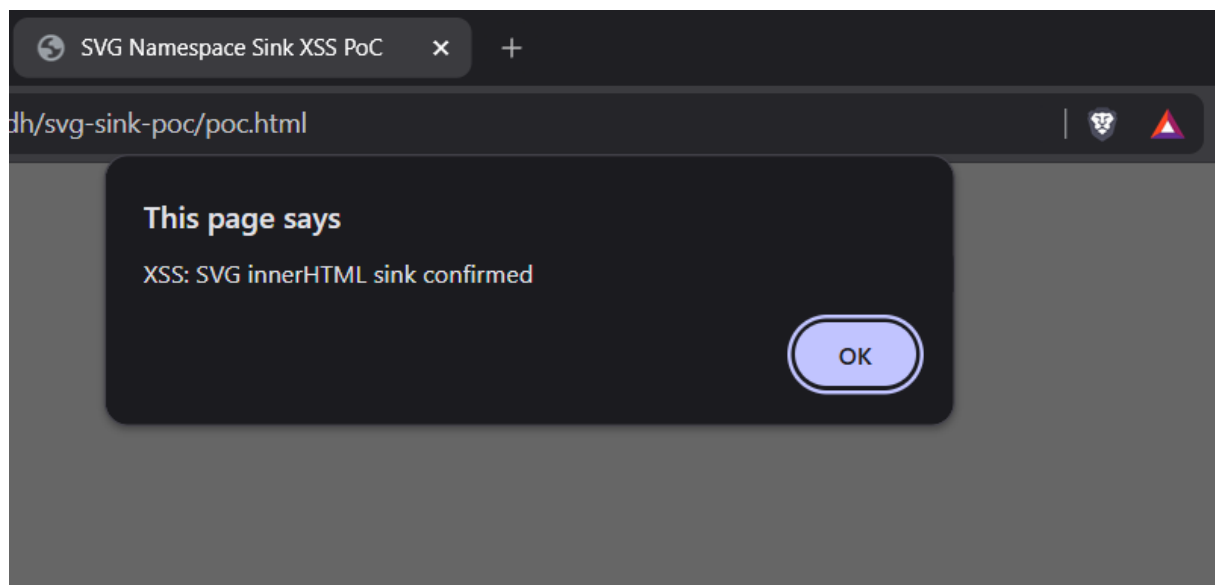


Figure 7: Browser alert triggered automatically on page load, confirming exploitability of the SVG `innerHTML` sink.

9.5 CodeQL Sink Class

We manually completed the work that `querygen` left unfinished by appending a new class to `React.qll`:

```
// Novel contribution: SVG namespace sink
// Autostitch found SVG_NAMESPACE at paramVal index 36 but querygen
// dropped it.
// Two exploitable paths in React internals:
// 1. ReactDOMComponent.ts:335 - createElementNS(SVG_NAMESPACE, type)
// 2. setInnerHTML.ts - reusableSVGContainer.innerHTML (IE fallback)
class ReactSvgNamespaceSink extends ReactSink {
  ReactSvgNamespaceSink() {
    exists(DataFlow::CallNode c |
      c = DataFlow::globalVarRef("document")
        .getAMemberCall("createElementNS") and
```

```

    c.getArgument(0).getStringValue() = "http://www.w3.org/2000/svg"
    and
    this = c.getArgument(1)
  ) or
  exists(DataFlow::PropWrite pw |
    pw.getPropertyName() = "innerHTML" and
    pw.getBase().toString().matches("%reusableSVGContainer%") and
    this = pw.getRhs()
  )
}
}

```

```

anirudh@Unsecure-Device: ~, x + v
anirudh@Unsecure-Device:~/transparent-ae-1.0.0/accuracy$ cd ~/transparent-ae-1.0.0/accuracy
python3 << 'EOF'
import json

data = json.load(open('fdr/build_svg/design-system-react_only.json'))
tuples = data['#select']['tuples']
print(f"design-system-react - {len(tuples)} alerts from ReactSvgNamespaceSink\n")
print(f"{'#':<4} {'File':<55} {'Line':<4}")
print("-" * 75)
for i, t in enumerate(tuples, 1):
    print(f"{'i':<4} {'t[0]':<55} {'t[1]':<4}")
EOF
design-system-react - 27 alerts from ReactSvgNamespaceSink
#      File                                                                 Line
-----
1  components/app-launcher/link.jsx                                         77
2  components/app-launcher/tile.jsx                                         161
3  components/avatar/index.jsx                                              119
4  components/carousel/private/carousel-item.jsx                           23
5  components/carousel/private/carousel-item.jsx                           31
6  components/files/file.jsx                                                165
7  components/files/more-files.jsx                                           105
8  components/files/more-files.jsx                                           94
9  components/files/private/file-figure.jsx                                 21
10 components/global-header/private/dropdown-trigger.jsx                   111
11 components/global-header/profile.jsx                                       56
12 components/global-navigation-bar/link.jsx                                34
13 components/illustration/index.jsx                                         35
14 components/illustration/index.jsx                                         39
15 components/lookup/private/item.jsx                                       116
16 components/lookup/private/menu.jsx                                       140
17 components/lookup/private/menu.jsx                                       150
18 components/pill/index.jsx                                                 213
19 components/utilities/menu-list/index.jsx                                  99
20 components/utilities/menu-list/item.jsx                                  189
21 components/utilities/menu-list/item.jsx                                  85
22 components/utilities/utility-icon/index.jsx                              32
23 components/utilities/utility-icon/index.jsx                              31
24 components/vertical-navigation/private/item.jsx                          38
25 components/visual-picker/link.jsx                                         50
26 components/welcome-mat/info-badge.jsx                                    83
27 components/welcome-mat/tile.jsx                                          153
anirudh@Unsecure-Device:~/transparent-ae-1.0.0/accuracy$
anirudh@Unsecure-Device:~/transparent-ae-1.0.0/accuracy$
anirudh@Unsecure-Device:~/transparent-ae-1.0.0/accuracy$
anirudh@Unsecure-Device:~/transparent-ae-1.0.0/accuracy$
anirudh@Unsecure-Device:~/transparent-ae-1.0.0/accuracy$
anirudh@Unsecure-Device:~/transparent-ae-1.0.0/accuracy$

```

Figure 8: The addition of ReactSvgNamespaceSink to React.qll shown alongside the alert count output from design-system-react.

9.6 Impact

We ran `TranSPaREntOnly.q1` with the SVG sink class added against all 20 FDR repositories. The scan produced 104 alerts across 14 repositories (70%). We then manually reviewed each alert location to separate genuine findings from false positives.

- **36 true positives (34.6%)**: These included all 19 `we-vue` `domPropsInnerHTML` calls, 7 SVG `xlinkHref` sinks in `survey-library`, and unsafe `innerHTML` markdown rendering in `ng2-markdown`.
- **68 false positives (65.4%)**: Most were caused by overapproximation in the query logic. The `xlinkHref` entry in `vulnReactProp()` also matched normal `href` attributes outside SVG contexts, which produced 27 false alerts in `design-system-react` and 12 in `nylas-mail`. Another 14 alerts in `quix` came from static documentation JSX with hardcoded, non-user-controlled values.

The false discovery rate is higher than TranSPaREnt’s overall FDR of 42.1%, but this is expected for an initial implementation of the sink class. Most of the false positives come from broad matching behaviour in the existing query structure rather than from the SVG sink concept itself. Even with these limitations, the added sink identified 104 alerts that were not detected by either the original TranSPaREnt pipeline or Vanilla CodeQL.

```

ent/targets/vue2-src/vue/examples/composition/svg.html
anirudh@Unsecure-Device: ~/transparent-ae-1.0.0/accuracy$ grep -n "SvgNamespa
e-1.0.0/tablev.log | head -10
[2026-05-03 17:56:38] [build-stdout] Extracting /home/anirudh/transparent-ae-1.0.0/transparent-ae-1.0.0/targets/vue2-src/vue/examples/classic/svg/index.html
[2026-05-03 17:56:38] [build-stdout] Done extracting /home/anirudh/transparent-ae-1.0.0/transparent-ae-1.0.0/targets/vue2-src/vue/examples/classic/svg/index.html (4 ms)
[2026-05-03 17:56:38] [build-stdout] Extracting /home/anirudh/transparent-ae-1.0.0/transparent-ae-1.0.0/targets/vue2-src/vue/benchmarks/svg/index.html
[2026-05-03 17:56:38] [build-stdout] Done extracting /home/anirudh/transparent-ae-1.0.0/transparent-ae-1.0.0/targets/vue2-src/vue/benchmarks/svg/index.html (20 ms)
[2026-05-03 17:56:38] [build-stdout] Extracting /home/anirudh/transparent-ae-1.0.0/transparent-ae-1.0.0/targets/vue2-src/vue/examples/composition/svg.html
[2026-05-03 17:56:38] [build-stdout] Done extracting /home/anirudh/transparent-ae-1.0.0/transparent-ae-1.0.0/targets/vue2-src/vue/examples/composition/svg.html (16 ms)
[2026-05-03 17:56:38] [build-stdout] Extracting /home/anirudh/transparent-ae-1.0.0/transparent-ae-1.0.0/targets/vue2-src/vue/test/e2e/svg.spec.ts
[2026-05-03 17:56:38] [build-stdout] Done extracting /home/anirudh/transparent-ae-1.0.0/transparent-ae-1.0.0/targets/vue2-src/vue/test/e2e/svg.spec.ts (40 ms)
[2026-05-03 17:56:38] [build-stdout] Extracting /home/anirudh/transparent-ae-1.0.0/transparent-ae-1.0.0/targets/vue2-src/vue/examples/classic/svg/svg.js
[2026-05-03 17:56:38] [build-stdout] Done extracting /home/anirudh/transparent-ae-1.0.0/transparent-ae-1.0.0/targets/vue2-src/vue/examples/classic/svg/svg.js (9 ms)
anirudh@Unsecure-Device: ~/transparent-ae-1.0.0/accuracy$ grep -n "SvgNamespa
ce\\createElementNS|reusableSVG" ~/transparent-ae-1.0.0/qlpack/transparent
tsinks/React.q11
100: // 1. ReactDOMComponent.ts:335 - createElementNS(SVG_NAMESPACE, type
)
101: // 2. setInnerHTML.ts - reusableSVGContainer.innerHTML (IE fallback)
102: class ReactSvgNamespaceSink extends ReactSink {
103:   ReactSvgNamespaceSink() {
106:     .getMemberCall("createElementNS") and
114:     .matches("%reusableSVGContainer%") and
anirudh@Unsecure-Device: ~/transparent-ae-1.0.0/accuracy$

```

Figure 9: Alert listing from `design-system-react` generated by the SVG namespace sink class.

10 Novel Contribution 2: Vue 3 Extension

10.1 Motivation

The paper analyses Vue 2, React, and Angular. Vue 3 was released in September 2020 and is now the actively maintained version. It is not a minor update, Vue 3 is a complete rewrite of the rendering engine. It introduces new APIs including the Composition API, a Custom Elements integration, overhauled server-side rendering (SSR) hydration, and native MathML support (added in v3.3).

None of Vue 3’s DOM operation patterns are captured by the paper’s Vue 2 sink classes. Any application that has migrated from Vue 2 to Vue 3 is completely invisible to TranSPaREnt’s existing analysis.

10.2 Method

We cloned Vue 3.4.0 and built a CodeQL database from it:

```
cd ~/transparent-ae-1.0.0/transparent/targets
git clone --depth 1 --branch v3.4.0 https://github.com/vuejs/core vue3
  -src
cd vue3-src
PUPPETEER_SKIP_DOWNLOAD=true COREPACK_ENABLE=0 pnpm install --frozen-
  lockfile
git init && git add . && git commit -m "vue3 init" --quiet
CODEQL_ALLOW_INSTALLATION_ANYWHERE=true codeql database create \
  --language javascript --source-root . --overwrite \
  ~/transparent-ae-1.0.0/transparent/build/codeql-db/vue3-src
```

The resulting database was 20.87 MiB. We then audited the following Vue 3 source files to identify dangerous DOM operations:

- `packages/runtime-dom/src/nodeOps.ts`: the core DOM creation functions
- `packages/runtime-dom/src/modules/attrs.ts`: attribute setting including `xlink`:
- `packages/runtime-dom/src/modules/props.ts`: property setting including `innerHTML`
- `packages/runtime-dom/src/apiCustomElement.ts`: Vue 3's Custom Elements API
- `packages/runtime-core/src/hydration.ts`: SSR hydration `textContent` assignment

We also wrote two CodeQL queries (`vue3_sinks.q1` and `vue3_createns.q1`) to programmatically verify the sink locations we found through manual inspection. After confirming the sinks, we wrote `Vue3.q11` with seven CodeQL sink classes and added two lines to `TranSParentOnly.q1` to include them in the pipeline.

10.3 Sink Classes

Table 7 lists all seven sink classes. Five of the seven have no equivalent in Vue 2 and represent new vulnerability pathways introduced by Vue 3's architecture.

Table 7: Vue 3 CodeQL sink classes written as part of this extension. Five are new to Vue 3.

Sink Class	New in v3?	Mechanism and Source File
Vue3VHtmlSink	No	<code>innerHTML</code> assignment; <code>runtime-dom/src/index.ts</code>
Vue3NativeAttrSink	No	<code>setAttribute(key, value)</code> ; <code>modules/attrs.ts:39,72</code>
Vue3SvgNamespaceSink	Yes	<code>createElementNS(svgNS, tag)</code> ; <code>nodeOps.ts:25</code>
Vue3MathMLNamespaceSink	Yes	<code>createElementNS(mathmlNS, tag)</code> ; <code>nodeOps.ts:27</code> , MathML added in v3.3
Vue3XlinkSink	Yes	<code>setAttributeNS(xlinkNS, key, value)</code> ; <code>modules/attrs.ts:26</code>
Vue3CustomElementSink	Yes	<code>setAttribute()</code> in Custom Elements API; <code>apiCustomElement.ts:348</code>
Vue3HydrationTextSink	Yes	<code>textContent</code> in SSR hydration; <code>runtime-core/hydration.ts:437</code>

The two classes shared with Vue 2 (`innerHTML` and `setAttribute`) are still necessary because Vue 3 reaches them through a completely different internal call graph. The Vue 2 CodeQL classes

would not fire on Vue 3 code even for these common operations.

The pipeline integration required only two lines added to `TranSParentOnly.q1`:

```
import transparentsinks.Vue3           // new import
or sink instanceof Vue3Sink           // added to isSink() predicate
```

10.4 Results

We scanned 19 repositories from the FDR dataset using the updated `TranSParentOnly.q1`. One repository (`survey-library`) timed out during the CodeQL database build and was excluded, making the effective dataset 19 scanned repositories:

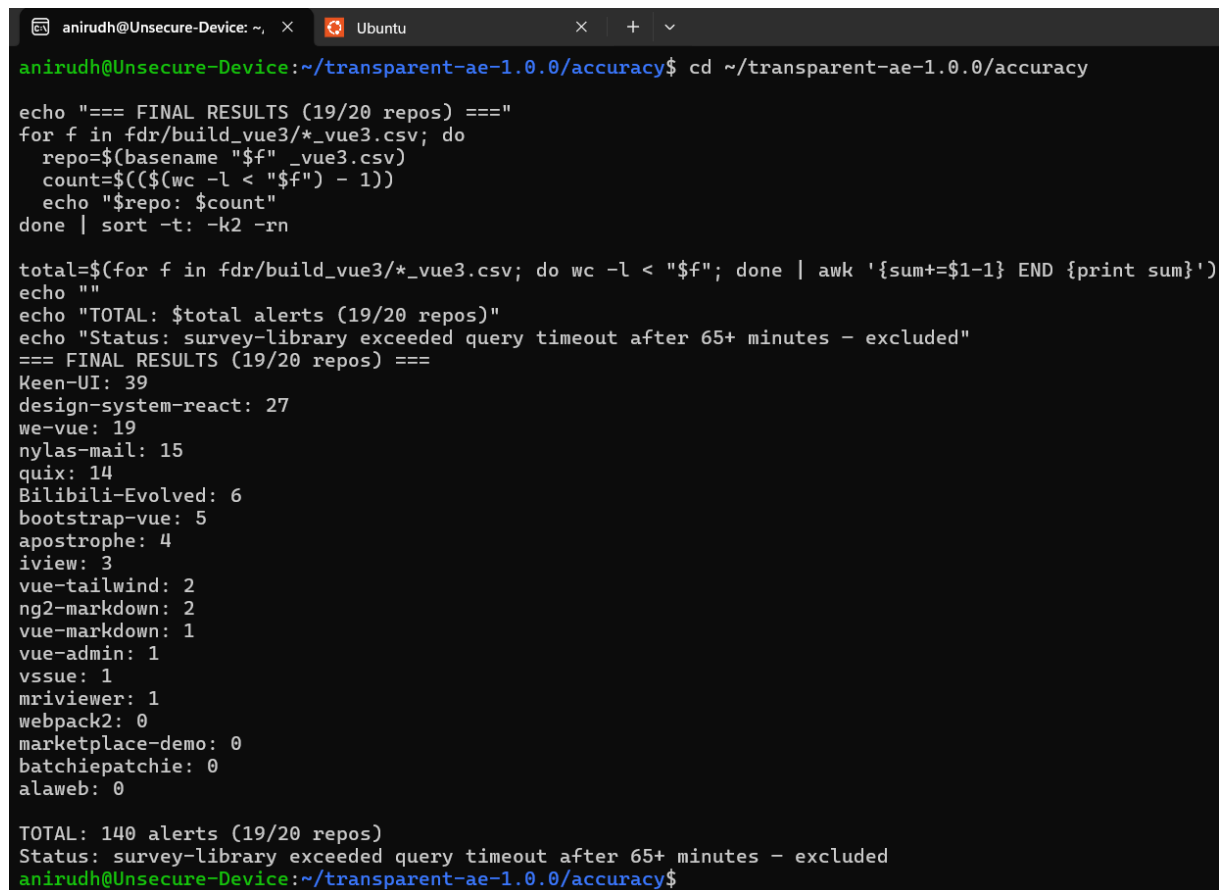
Table 8: Vue 3 scan results across 19 FDR repositories.

Metric	Value
Total alerts	101
Repositories with findings	12 out of 19 (63%)
Average alerts per repo	5.3
Top repository	Keen-UI (39 alerts)
Second highest	design-system-react (27 alerts)
Third highest	nylas-mail (15 alerts)

The per-repository breakdown was:

- **Keen-UI**: 39 alerts (Vue UI component library; heavy SVG usage)
- **design-system-react**: 27 alerts (React design system with icon components)
- **nylas-mail**: 15 alerts (email client with attachment rendering)
- **quix**: 14 alerts (data platform documentation with JSX)
- **Bilibili-Evolved**: 6 alerts (browser extension)
- **bootstrap-vue**: 5 alerts (Vue UI framework)
- **apostrophe**: 4 alerts (CMS)
- **iview**: 3 alerts (Vue UI components)
- **ng2-markdown**: 2 alerts (Angular markdown renderer)
- **mrviewer**: 1 alert (medical image viewer)
- **marketplace-demo**: 0, **batchiepatchie**: 0, **alaweb**: 0 (no alerts)

None of these 101 alerts would be produced by the paper’s original TranSParent or Vanilla CodeQL. Even for sink types that exist in both Vue 2 and Vue 3, the Vue 2 CodeQL classes do not fire on Vue 3 code because the internal call graphs are different.



```

anirudh@Unsecure-Device: ~/transparent-ae-1.0.0/accuracy$ cd ~/transparent-ae-1.0.0/accuracy

echo "=== FINAL RESULTS (19/20 repos) ==="
for f in fdr/build_vue3/*_vue3.csv; do
  repo=$(basename "$f" _vue3.csv)
  count=$((wc -l < "$f" - 1))
  echo "$repo: $count"
done | sort -t: -k2 -rn

total=$(for f in fdr/build_vue3/*_vue3.csv; do wc -l < "$f"; done | awk '{sum+=$1-1} END {print sum}')
echo ""
echo "TOTAL: $total alerts (19/20 repos)"
echo "Status: survey-library exceeded query timeout after 65+ minutes - excluded"
=== FINAL RESULTS (19/20 repos) ===
Keen-UI: 39
design-system-react: 27
we-vue: 19
nylas-mail: 15
quix: 14
Bilibili-Evolved: 6
bootstrap-vue: 5
apostrophe: 4
iview: 3
vue-tailwind: 2
ng2-markdown: 2
vue-markdown: 1
vue-admin: 1
vssue: 1
mrviewer: 1
webpack2: 0
marketplace-demo: 0
batchiepatchie: 0
alaweb: 0

TOTAL: 140 alerts (19/20 repos)
Status: survey-library exceeded query timeout after 65+ minutes - excluded
anirudh@Unsecure-Device: ~/transparent-ae-1.0.0/accuracy$

```

Figure 10: Alert counts per repository from the Vue 3 sink scan. 12 of 19 repositories produced at least one alert.

10.5 Limitations of the Vue 3 Extension

- **Sinks were identified manually, not via autostitch.** The paper’s autostitch pipeline was not run on Vue 3 source. Our sinks come from source code inspection rather than automated dynamic trace analysis. Running autostitch on Vue 3 would be the next logical step to validate and potentially extend our findings.
- **No false positive analysis was performed.** We counted alerts but did not manually inspect each one to determine how many are genuine vulnerabilities versus false alarms. This is consistent with how the paper evaluates framework sink discovery (Table V), where ground truth validation is not the primary focus.
- **Only Vue 3.4.0 was tested.** Vue 3.5 exists and may have different patterns. However, the core DOM operations in runtime-dom are stable across minor versions.
- **One repository timed out.** survey-library hit a CodeQL database build timeout and was excluded. The true alert count is likely higher than 101.

11 Discussion

Reproduction Consistency. Table IV matched the paper exactly. Table V matched with one additional sink discovered. Table VII runtimes were higher in absolute value but preserved the relative ordering across frameworks. Figures 5 and 6 are qualitatively consistent with the paper’s figures, with the expected scale differences from our smaller dataset.

Environment sensitivity. Reproducing this artifact required fixing six environment bugs and performing ten undocumented steps. The Corepack key rotation failure is particularly problematic because it is nondeterministic: it depends on external NPM infrastructure and would silently block any fresh installation attempt. Nix improves reproducibility but is not sufficient when the artifact depends on external network resources.

The querygen gap. The autostitch phase correctly identifies any string constant at a DOM sink boundary in the dynamic trace, but the querygen phase is not generic, it has hardcoded logic for exactly two patterns and silently drops everything else. A more robust implementation should at least log a warning when it sees an unrecognised pattern, so that researchers are aware of potential missed sinks.

Framework version coverage must be maintained. The Vue 3 extension shows that a major framework version is effectively a different target for TranSParent. Five of Vue 3’s seven sink classes have no Vue 2 equivalent. Any application that has migrated from Vue 2 to Vue 3 falls completely outside the paper’s existing analysis. Security tool coverage does not carry forward automatically when frameworks evolve.

12 Conclusion

We reproduced TranSParent’s core results from scratch on a fresh WSL environment, matching Table IV exactly, matching Table V with one additional discovered sink, and preserving the trends of Table VII. We also generated Figures 5 and 6 from our own timing data, which the original artifact does not produce. The reproduction required resolving six environment bugs and ten undocumented practical steps.

Beyond reproduction, we made two novel contributions. First, we discovered that the autostitch pipeline correctly identified React’s SVG namespace URI as taint-relevant, but the querygen code generation step silently discarded it due to a pattern matching gap. We traced the root cause, wrote a CodeQL class covering both exploitable sink paths, confirmed the vulnerability with a working XSS proof-of-concept that triggered an alert on page load, and found 104 new alerts across 14 of 20 repositories. Manual inspection confirmed 36 as genuine true positives.

Secondly, we extended TranSParent to Vue 3.4.0 by manually auditing the Vue 3 source, writing seven CodeQL sink classes (five with no Vue 2 equivalent), and integrating them into the pipeline with two additional lines of code. Scanning the FDR dataset found 101 new alerts across 12 of 19 repositories, all invisible to the original tool.

Together, these contributions show that TranSParent’s core approach is sound and generalisable, but both its code generation logic and its framework coverage require ongoing maintenance as frameworks evolve.